

CLOUD COMPUTING: DISTRIBUTED SYSTEMS

Module 6: Distributed Systems & Cloud Native

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

- 1 Distributed Systems Fundamentals
- 2 Communication Patterns
- 3 Microservices Architecture
- 4 Cloud Native Principles



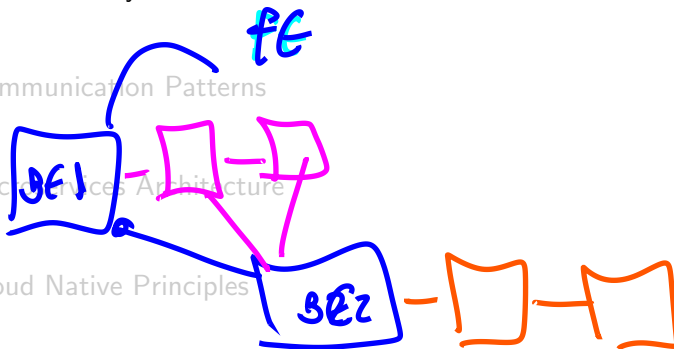
Outline

1 Distributed Systems Fundamentals

2 Communication Patterns

3 Microservices Architecture

4 Cloud Native Principles



Centralized vs. Distributed

Centralized

- Single point of control
- Easy consistency
- Scales poorly
- Single point of failure

Distributed

- Multiple independent components
- Complex consistency
- Scales horizontally
- No single point of failure

typical - old school

The Trade-off

Distributed systems are more resilient but harder to reason about.



Centralized vs. Distributed

Centralized

- Single point of control
- Easy consistency
- Scales poorly
- Single point of failure

Distributed

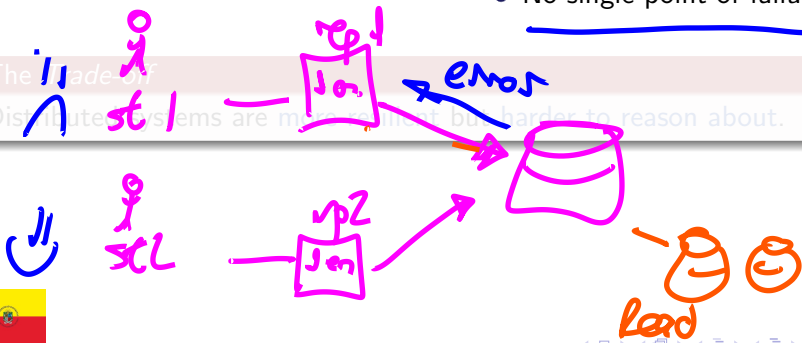
- Multiple independent components
- Complex consistency
- Scales horizontally
- No single point of failure

Docter

KS

The Trade Off

Distributed systems are more resilient but harder to reason about.



Centralized vs. Distributed

Centralized

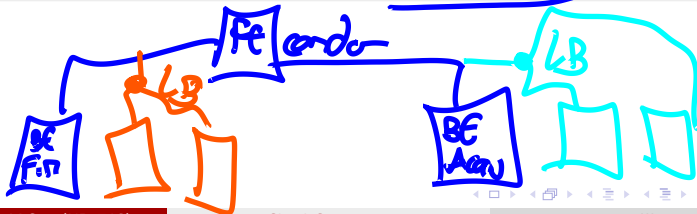
- Single point of control
- Easy consistency
- Scales poorly
- Single point of failure

Distributed

- Multiple independent components
- Complex consistency
- Scales horizontally
- No single point of failure

The *Trade-off*

Distributed systems are more resilient but harder to reason about.

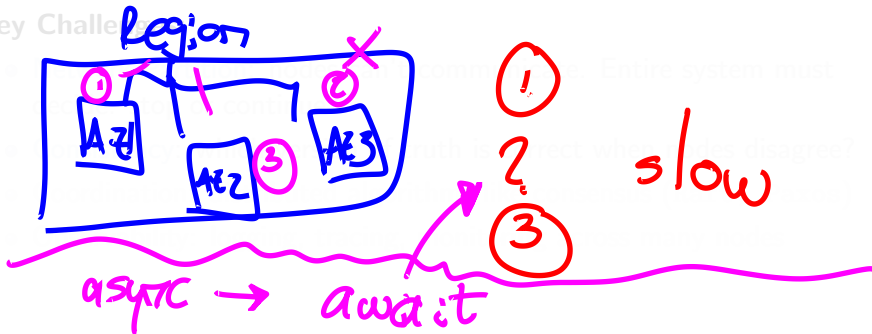


Challenges of Distributed Systems

The Hard Truth

Distributed systems must handle: latency, partial failures, and asynchrony. *There is no magic solution.*

Key Challenges



Challenges of Distributed Systems

The *Hard Truth*

Distributed systems must handle: **latency**, **partial failures**, and **asynchrony**. *There is no magic solution.*

Key Challenges:

- Network Partition: nodes can't communicate. Entire system must decide: stop or continue?
- Consistency: which version of truth is correct when nodes disagree?
- Coordination: distributed algorithms like consensus (Raft, Paxos)
- Observability: logging, tracing, monitoring across many nodes



Challenges of Distributed Systems

The *Hard Truth*

Distributed systems must handle: **latency**, **partial failures**, and **asynchrony**. *There is no magic solution.*

Key Challenges:

- **Network Partition**: nodes can't communicate. Entire system must decide: stop or continue?
- **Consistency**: which version of truth is correct when nodes disagree?
- **Coordination**: distributed algorithms like consensus (Raft, Paxos)
- **Observability**: logging, tracing, monitoring across many nodes



Challenges of Distributed Systems

The *Hard Truth*

Distributed systems must handle: **latency**, **partial failures**, and **asynchrony**. *There is no magic solution.*

Key Challenges:

- **Network Partition**: nodes can't communicate. Entire system must decide: stop or continue?
- **Consistency**: which version of truth is correct when nodes disagree?
- **Coordination**: distributed algorithms like consensus (Raft, Paxos)
- **Observability**: logging, tracing, monitoring across many nodes



Challenges of Distributed Systems

The *Hard Truth*

Distributed systems must handle: **latency**, **partial failures**, and **asynchrony**. *There is no magic solution.*

Key Challenges:

- **Network Partition**: nodes can't communicate. Entire system must decide: stop or continue?
- **Consistency**: which version of truth is correct when nodes disagree?
- **Coordination**: distributed algorithms like consensus (Raft, Paxos)
- **Observability**: logging, tracing, monitoring across many nodes

query

process flow



Outline

- 1 Distributed Systems Fundamentals
- 2 **Communication Patterns**
- 3 Microservices Architecture
- 4 Cloud Native Principles

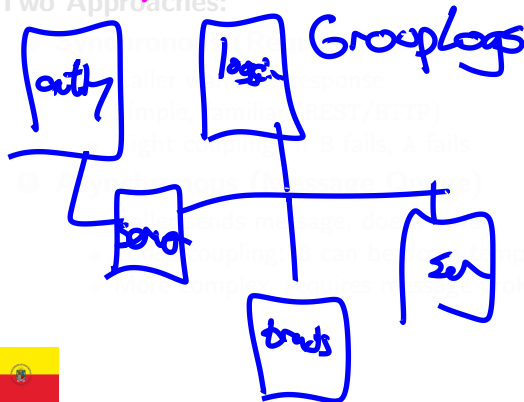


Service-to-Service Communication *cloud*

The Question

How should Service A talk to Service B? Synchronously? Asynchronously?
Reliably? Quickly?

Two Approaches:



Service-to-Service Communication

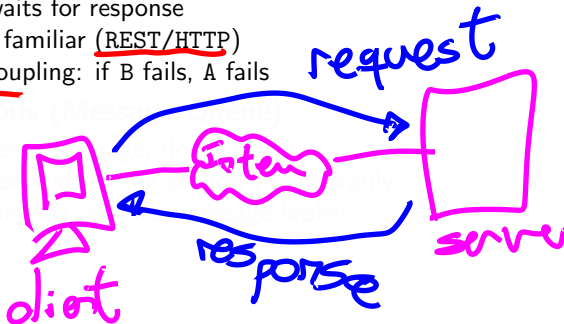
The Question

How should Service A talk to Service B? Synchronously? Asynchronously? Reliably? Quickly?

Two Approaches:

① Synchronous (Request-Response)

- Caller waits for response
- Simple, familiar (REST/HTTP)
- Tight coupling: if B fails, A fails



Service-to-Service Communication

The Question

How should Service A talk to Service B? Synchronously? Asynchronously? Reliably? Quickly?

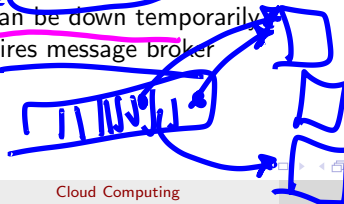
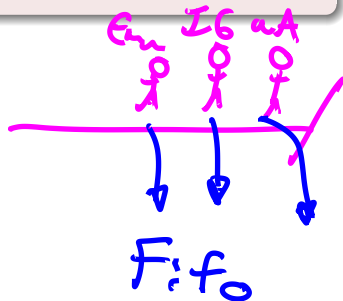
Two Approaches:

1 Synchronous (Request-Response)

- Caller waits for response
- Simple, familiar (REST/HTTP)
- Tight coupling: if B fails, A fails

2 Asynchronous (Message Queue)

- Caller sends message, doesn't wait
- Loose coupling: B can be down temporarily
- More complex, requires message broker



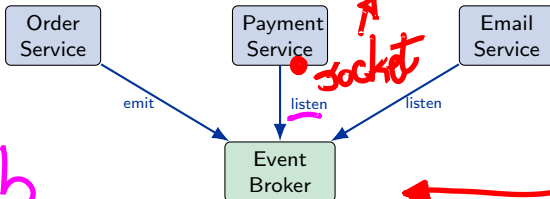
Event-Driven Architecture

→ serverless

Core Idea

Instead of services calling each other directly, they emit **events**. Other services subscribe to events they care about.

observer



pub/sub

Benefits:

daemon

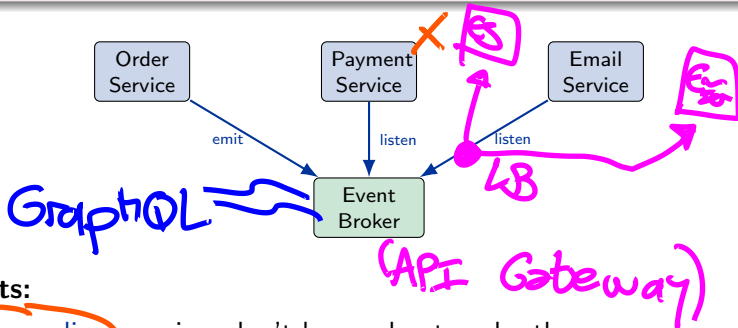
queue
↓
async



Event-Driven Architecture

Core Idea

Instead of services calling each other directly, they emit events. Other services subscribe to events they care about.



Benefits:

- Decoupling: services don't know about each other
- Scalability: add new listeners without changing producer



Outline

- 1 Distributed Systems Fundamentals
- 2 Communication Patterns
- 3 Microservices Architecture**
- 4 Cloud Native Principles



Monolith vs. Microservices

Monolith

→ my first project

- Single codebase
- Single deployment
- Shared database
- Tight coupling
- Simpler to start

Microservices

- Many codebases
- Independent deployment
- Database per service
- Loose coupling
- Complex operations

Golden Rule

Use microservices only when you have the **operational maturity** and **team size** to handle distributed complexity. Otherwise, keep a modular monolith.



Monolith vs. Microservices

Monolith

- Single codebase
- Single deployment
- Shared database
- Tight coupling
- Simpler to start

Microservices

- Many codebases
- Independent deployment
- Database per service
- Loose coupling
- Complex operations

Golden Rule

Use microservices only when you have the **operational maturity** and **team size** to handle distributed complexity. Otherwise, keep a modular monolith.



Monolith vs. Microservices

Monolith

- Single codebase
- Single deployment
- Shared database
- Tight coupling
- Simpler to start

Microservices

- Many codebases
- Independent deployment
- Database per service
- Loose coupling
- Complex operations

Golden Rule

Use microservices only when you have the operational maturity and team size to handle distributed complexity. Otherwise, keep a modular monolith.

un

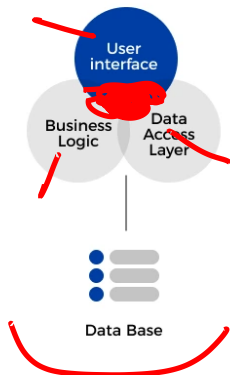
(A)

B



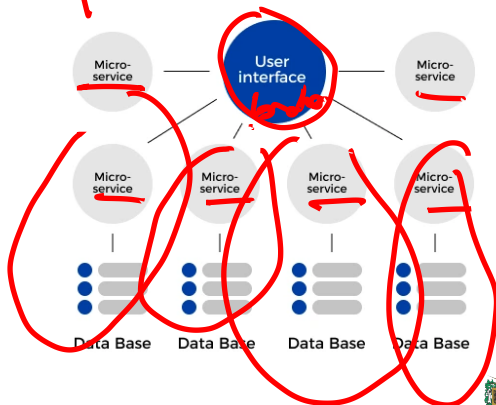
Monolithic vs. Microservices Architectures

MONOLITHIC ARCHITECTURE



each 'service' has an independent hardware

MICROSERVICE ARCHITECTURE



Microservices Architecture Pattern



Microservices Architecture
on Google Cloud

cloud → agnostics
#GCPsketchnotes @PVERGADIA @THECLOUDGIRL.DEV 04.12.2021
SLA

What are the benefits of microservices?

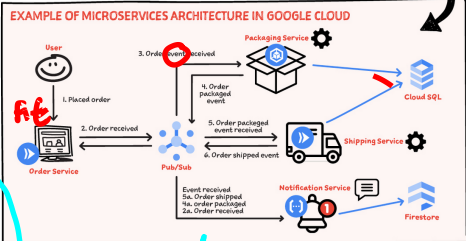
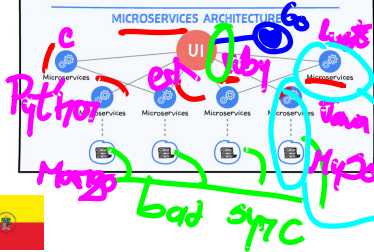


What's the difference between monolith & microservices architecture?

MONOLITHIC ARCHITECTURE



MICROSERVICES ARCHITECTURE

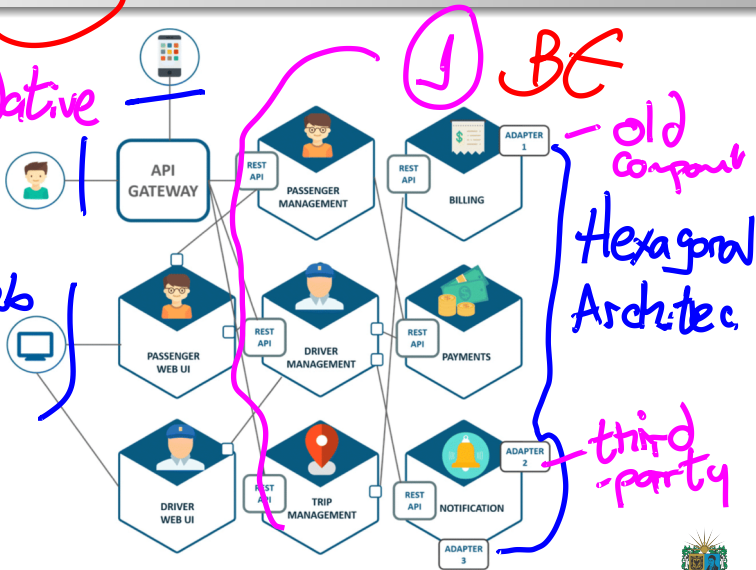


Case Study: Uber Microservices

React-Native

F.E

web



Source: Kappagantula 2018



Outline

- 1 Distributed Systems Fundamentals
- 2 Communication Patterns
- 3 Microservices Architecture
- 4 Cloud Native Principles**



What is Cloud Native?

The Philosophy

Applications built and deployed to take advantage of cloud elasticity, containerization, APIs, and observability.

Core Characteristics

micro-services

- D.D.D.
- Hexagonals
- web services

The Twelve-Factor App

A methodology for building SaaS applications. See 12 factors at

production
by final users

↳ after deploy,
what happens?

- metrics
 - request / hour
 - logs errors
 - response time warning
 - most visited section



What is Cloud Native?

The Philosophy

Applications built and deployed to take advantage of cloud elasticity, containerization, APIs, and observability.

Core Characteristics:

- **Containerized:** packaged with dependencies
- Stateless: session data external to instances
- Automated deployment, scaling, recovery by platform
- Observable: logs, metrics, traces everywhere
- Resilient: designed to handle failures gracefully

Docker

The Twelve-Factor App

A methodology for building SaaS applications. See: 12factor.net



What is Cloud Native?

The Philosophy

Applications built and deployed to take advantage of cloud elasticity, containerization, APIs, and observability.

Core Characteristics:

- **Containerized**: packaged with dependencies
- **Stateless**: session data external to instances
- Automated: deployment, scaling, recovery by platform
- Observable: logs, metrics, traces everywhere
- Resilient: designed to handle failures gracefully

The Twelve-Factor App

A methodology for building SaaS applications. See: 12factor.net



What is Cloud Native?

The Philosophy

Applications built and deployed to take advantage of cloud elasticity, containerization, APIs, and observability.

Core Characteristics:

- **Containerized**: packaged with dependencies
- **Stateless**: session data external to instances
- **Automated**: deployment, scaling, recovery by platform
- **Observable**: logs, metrics, traces everywhere
- **Resilient**: designed to handle failures gracefully

The Twelve-Factor App

A methodology for building SaaS applications. See: 12factor.net



What is Cloud Native?

The Philosophy

Applications built and deployed to take advantage of cloud elasticity, containerization, APIs, and observability.

Core Characteristics:

- **Containerized**: packaged with dependencies
- **Stateless**: session data external to instances
- **Automated**: deployment, scaling, recovery by platform
- **Observable**: logs, metrics, traces everywhere
- **Resilient**: designed to handle failures gracefully

The Twelve-Factor App

A methodology for building SaaS applications. See: 12factor.net



What is Cloud Native?

The Philosophy

Applications built and deployed to take advantage of cloud elasticity, containerization, APIs, and observability.

Core Characteristics:

- **Containerized**: packaged with dependencies
- **Stateless**: session data external to instances
- **Automated**: deployment, scaling, recovery by platform
- **Observable**: logs, metrics, traces everywhere
- **Resilient**: designed to handle failures gracefully

The Twelve-Factor App

A methodology for building SaaS applications. See: 12factor.net



What is Cloud Native?

The Philosophy

Applications built and deployed to take advantage of cloud elasticity, containerization, APIs, and observability.

Core Characteristics:

- **Containerized**: packaged with dependencies
- **Stateless**: session data external to instances
- **Automated**: deployment, scaling, recovery by platform
- **Observable**: logs, metrics, traces everywhere
- **Resilient**: designed to handle failures gracefully

The Twelve-Factor App

A methodology for building SaaS applications. See: 12factor.net



Outline

- Use declarative formats (Configs)

YAML

1 Distributed Systems Fundamentals

- Clean contract

2 Communication Patterns

- automatic deployment to cloud

→ GitHub Actions

3 Microservices Architecture

↘ minimize divergence

4 Cloud Native Principles

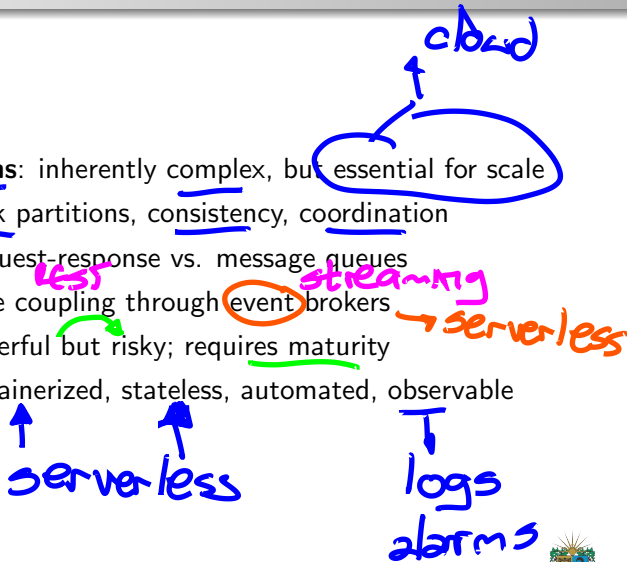
- scale strategy

(Prod) — Dev



Recap: Module 6

- Distributed Systems: inherently complex, but essential for scale
- Challenges: network partitions, consistency, coordination
- Sync vs Async: request-response vs. message queues
- Event-Driven: loose coupling through event brokers
- Microservices: powerful but risky; requires maturity
- Cloud Native: containerized, stateless, automated, observable



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing>

